Rapidly-Exploring Random Trees (RRT)

Contents

1. Overview 2
2. Growing a tree from random samples: RRT 3
   - 2.1 One step: sample, nearest node, steer, check 3
   - 2.2 The tree grows to the door 5
   - 2.3 Why random samples explore fast 6
   - 2.4 The algorithm and the goal bias 7
   - 2.5 The route is long, and stays long 8
3. Shortening the route: RRT* and asymptotic optimality 9
   - 3.1 Choose the parent, then rewire 9
   - 3.2 The route shrinks as the tree grows 11
   - 3.3 Asymptotic optimality 12
4. Two trees that meet: RRT-Connect 14
5. Summary 16
6. Sources 17
7. Key terms 18

Where this fits:

- **Builds on:** Backward and bidirectional search; Checking a path segment; Metric; Nearest neighbours with wrap-around.
- **Leads to:** Kinodynamic RRT.
- **Compare with:** Randomized potential fields; Probabilistic roadmap (PRM).

1. Overview

Key point: A rapidly-exploring random tree grows from the start by drawing random configurations and stepping the nearest node of the tree a short way towards each one; it needs no grid, only collision checks and nearest neighbours. Plain RRT finds a long route quickly; RRT* keeps rewiring the tree so the route shrinks towards the shortest; RRT-Connect grows a tree from each end and finds a route sooner.

So far the robot has met the room's furniture with a potential field (G-2740), a height it steps down from wherever it is. Under the table, at (4.0, 0.5), the robot stopped in a local minimum (G-1110) at (3.60, 0.59), and every fix had a price:

- random walks got it out, after a median of 46.1 m of driving where the shortest route is 4.55 m;
- a harmonic potential has no local minimum, but needs 29.2 million updates on a grid of the whole room first;
- a navigation function works only in a world made of disks.

Grids do not scale. The room's grid of points 0.05 m apart has 11,421 points; a robot that turns, with a heading every 1°, needs 4.1 million; an arm with six joints at 1° each needs 2.2×10^{15}. LaValle's example: 50 points along each of 10 axes is 50^{10} points, impossible to search (LaValle §5.4.3).

This Note grows a tree instead, from random samples, using only the two tests of chapter RO-04: is this configuration or segment free (collision checking), and which stored configuration is nearest (nearest neighbours). Figure 1 shows the three trees of this Note on the robot under the table; the red star is where the potential field left it.

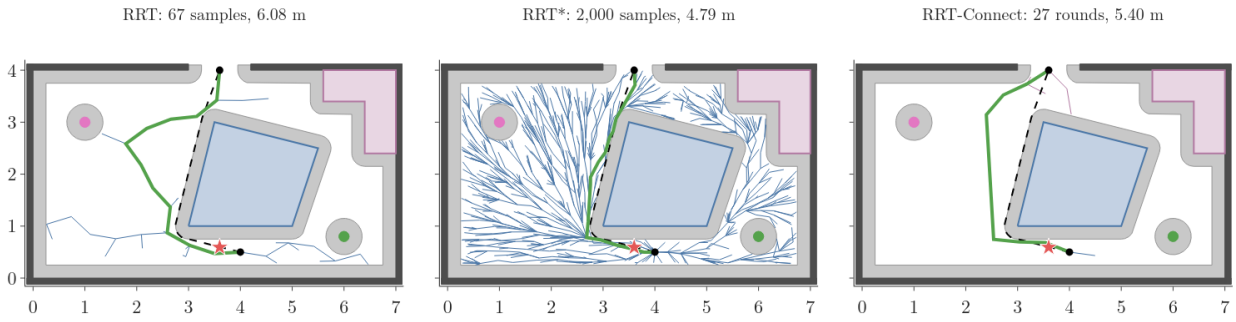


Figure 1: Three planners from under the table (4.0, 0.5) to the doorway centre (3.6, 4.0), in the disk robot's C-space (light grey: obstacles grown by 0.25 m). Red star: (3.60, 0.59), where gradient descent on the potential field stopped. Black dashed: the shortest route, 4.55 m. Left: RRT after 67 samples, route 6.08 m (green). Middle: RRT* after 2,000 samples, route 4.79 m. Right: RRT-Connect after 27 rounds, two trees (blue from the start, purple from the door), route 5.40 m

The opening question of this Note: **how can the robot under the table find a route to the door without a grid, how close can that route get to the shortest, 4.55 m, and how soon can a first route appear?** We answer it in steps:

- one random sample at a time: the rapidly-exploring random tree (Section 2);
- a tree that keeps shortening its routes: RRT* (Section 3);
- two trees that run to meet each other: RRT-Connect (Section 4).

Every number below is computed in the notebook `RO-051-rrt.ipynb`.

2. Growing a tree from random samples: RRT

> **Key point:** Each round, RRT draws a random configuration, finds the tree's nearest node, steps from it at most 0.5 m towards the sample and keeps the new node if the step is free. From under the table it reached the door after 67 samples with a route of 6.08 m; over 200 runs the median is 65 samples and 6.07 m, a third longer than the shortest route.

2.1 One step: sample, nearest node, steer, check

The first part of the opening problem: the robot under the table must get somewhere without a grid and without being stuck. Picture throwing darts at a map of the room. Wherever a dart lands, the nearest place the robot can already reach sends out a short branch towards it. Darts land all over the room, so branches reach out in every direction, round the table as easily as along it.

The standard terms. A planner that never builds the obstacle region or a grid, but draws random configurations and tests them only with collision checks, is a **sampling-based planner** (G-2760) (LaValle ch.5; MR §10.5). The tree it grows here is a **rapidly-exploring**

random tree (G-2761), RRT for short (LaValle §5.5): a graph (G-2540) of free configurations, its vertices, which we call nodes, joined by free straight edges, with exactly one path from the start to every node. One round has four moves (MR §10.5.1, Algorithm 10.3; CMU 16-350 slides):

1. **Sample:** draw a configuration evenly at random over the room's floor.
2. **Nearest node:** find the tree's **nearest node** (G-2762), the node closest to the sample, by a nearest-neighbour search.
3. **Steer:** move from the nearest node towards the sample, at most a step of 0.5 m; this step is the **steer** (G-2763) function.
4. **Check:** test the new configuration and the edge to it; if both are free, add the node, with the nearest node as its parent (G-2576); if not, throw it away.

The first sample, on numbers. The tree starts as one node, the start (4.0, 0.5). The symbols:

- q_{rand} is the sample, (1.395, 0.354), in m;
- q_{near} is the tree's nearest node, here the start (4.0, 0.5), in m;
- η (eta) is the longest step, 0.5 m;
- $\|q_{\text{rand}} - q_{\text{near}}\|$ is the Euclidean distance (G-715) between them, in m;
- q_{new} is the new node, in m.

The sample is 2.605 m left of the start and 0.146 m below it:

$$q_{\text{rand}} - q_{\text{near}} = (-2.605,\ -0.146)$$

$$\|q_{\text{rand}} - q_{\text{near}}\| = \sqrt{2.605^2 + 0.146^2}$$

$$= 2.609 \text{ m}$$

The sample is farther than 0.5 m, so the step goes 0.5 m along that direction:

$$q_{\text{new}} = (4.0,\ 0.5) + \frac{0.5}{2.609}(-2.605,\ -0.146)$$

$$= (4.0,\ 0.5) + (-0.499,\ -0.028)$$

$$= (3.501,\ 0.472)$$

In symbols, steer moves at most η towards the sample, and stops on it if it is closer (LaValle §5.5.1; Karaman and Frazzoli 2011):

$$q_{\text{new}} = q_{\text{near}} + \frac{\eta}{\|q_{\text{rand}} - q_{\text{near}}\|} (q_{\text{rand}} - q_{\text{near}})$$

when the sample is farther than η. Otherwise the new node is the sample itself:

$$q_{\text{new}} = q_{\text{rand}}$$

The check is edge resolution (G-2679) with points at most 0.05 m apart. With $\lceil \cdot \rceil$ rounding up to a whole number, the number of steps along the 0.5 m edge is:

$$\text{steps} = \lceil 0.5/0.05 \rceil = 10$$

so 11 points along the 0.5 m edge, each a test of the robot's centre against the obstacles grown by 0.25 m. All 11 are free: the node (3.501, 0.472) joins the tree. The second sample, (3.215, 3.951), lands near the door; its nearest node is the new one, 3.49 m away, and the step of 0.5 m towards it ends at (3.46, 0.97), inside the table's grown band. Rejected. One round adds at most one short edge, so the tree needs many rounds to reach the door.

2.2 The tree grows to the door

Figure 2 plays the whole run. Samples 1 to 4 go stage by stage: the sample (orange star), its nearest node (circled), the steered edge with its 11 check points, then added (green) or thrown away (red cross). After that, one sample per frame. Watch the tree crawl along the strip under the table, then branch up the table's left side towards the door.

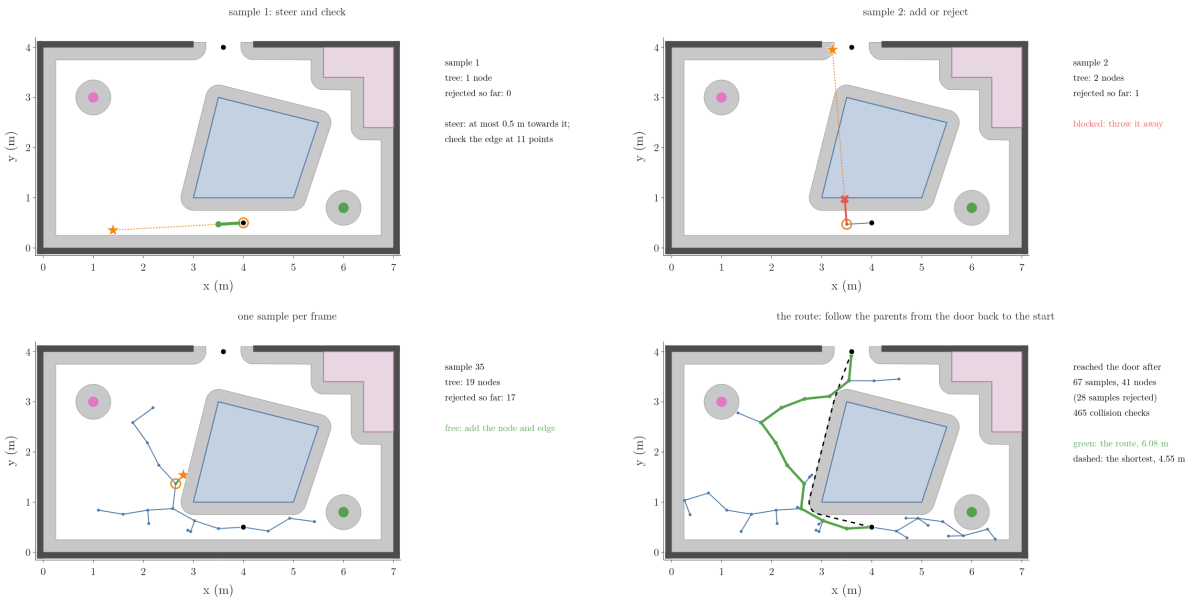


Figure 2: Animation: RRT growing from the start under the table (4.0, 0.5) to the doorway centre (3.6, 4.0), in the disk robot's C-space (light grey: obstacles grown by 0.25 m). Samples 1 to 4 stage by stage: the random sample (orange star), its nearest node (circled), a step of at most 0.5 m towards it checked at 11 points, then the node added (green) or rejected (red cross). Sample 1, (1.395, 0.354), is 2.609 m from the start and gives the node (3.501, 0.472); sample 2 lands near the door and its step ends inside the table's band. Then one sample per frame. Sample 67 is the goal itself; the last frame shows the tree of 41 nodes, the route through it, 6.08 m (green), and the shortest route, 4.55 m (dashed)

The tree reached the door at sample 67. Of the 67 samples, 28 were rejected, 17 of them in the first half of the run, while much of the tree was still in the 0.5 m strip under the table, where many steps hit the table or the wall. The tree has 41 nodes: the start, 39 added nodes and the door. The 465 collision checks are the whole cost: no grid, no field. The route is read off by following the parents from the door back to the start, as in A*: 13 edges, 6.08 m. So the robot that was stuck under the table has a route, which answers the first part of the opening problem.

2.3 Why random samples explore fast

Random walks freed the robot from the local minimum, but slowly (46.1 m of driving). An RRT is random too; why does it spread so much faster? Figure 3 grows two trees of 100 nodes each from under the table. Left: RRT, with no pull towards the door. Right: a random-walk tree, which picks a node of the tree at random and steps 0.5 m in a random direction (MR §10.5.1, Figure 10.17). The titles give the share of the free floor within 0.5 m, one step, of some node.

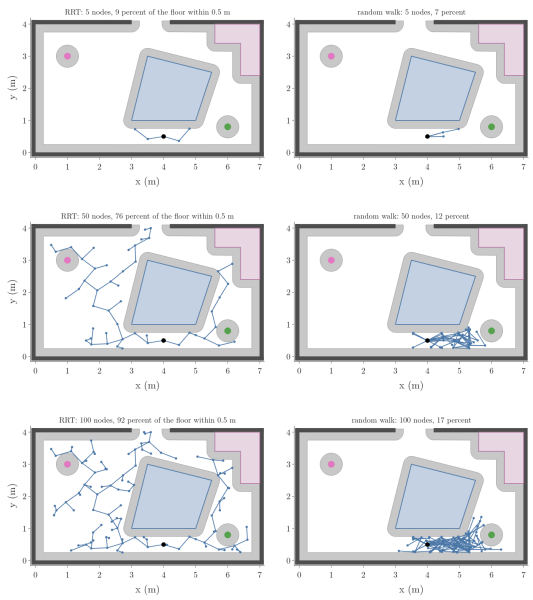


Figure 3: Animation: two trees grow from under the table (4.0, 0.5) to 100 nodes, a few nodes per frame. Left: RRT without goal bias, which reaches round both sides of the table and up to the door; after 100 nodes 92 percent of the free floor is within 0.5 m of a node in this run. Right: a random-walk tree, each new node 0.5 m from a randomly chosen node in a random direction; it stays in a tangle under the table, 17 percent

Over 20 runs, after 100 nodes, RRT covers 95 percent of the free floor and the random-walk tree 27 percent (notebook). The reason is which node grows. A random sample most often lands in a large empty region, and the node nearest to it is then one at the edge of the explored part, facing that empty region; so the tree grows outwards, towards the largest unexplored region (CMU 16-350 slides; LaValle §5.5.1). The random walk picks any node of the tree, and most nodes sit inside the explored tangle, so most new nodes land there too. The more empty floor lies around a node, the more often RRT extends it. Exploring fast does not yet mean reaching the door: nothing so far pulls the tree towards it.

2.4 The algorithm and the goal bias

Pure random samples would reach the door only by luck, so every so often the sample is the goal itself: with probability 0.05 here, 1 in 100 in LaValle §5.5.3, 0.1 in the CMU slides. Sampling the goal now and then is the **goal bias** (G-2764). Too strong a bias makes the tree greedy, running straight at the door like the randomized potential field (G-2744); too weak, and it rarely tries to finish (LaValle §5.5.3). In our run, sample 67 was such a goal sample, and the step from a node within 0.5 m of the door reached it.

The whole algorithm (LaValle §5.5; MR §10.5.1; CMU 16-350 slides):

1. Start the tree with the start configuration.
2. Repeat:
    1. draw a sample: the goal with probability 0.05, otherwise a random configuration;

2. find the tree's nearest node;
3. steer from it at most 0.5 m towards the sample;
4. if the new configuration and the edge to it are free, add the node with the nearest node as its parent;
5. if the new node is within 0.5 m of the goal and the edge to the goal is free, add the goal and stop.

3. Follow the parents from the goal back to the start: that is the route.

Three choices inside the loop shape any RRT (MR §10.5.1): the sampler (even over the floor here), the distance used to find the nearest node, and the way steer moves from one configuration towards another (a straight line here). For a robot with a heading, a configuration is (x, y, θ), and the distance is the weighted metric (G-2681) with the nearest-neighbour search with wrap-around (G-2685); the loop itself is unchanged.

What RRT guarantees. As the samples go on, the tree comes as close as we like to every free configuration, with probability 1, so if a route exists, RRT finds one eventually (LaValle §5.5.1; CMU 16-350 slides, for RRT-Connect).

2.5 The route is long, and stays long

The second part of the opening problem: how long is the route? Over 200 runs, RRT reached the door after a median of 65 samples, with a median route of 6.07 m against the shortest 4.55 m: a third longer, zig-zagging from node to node. Can it improve if we keep sampling and keep the best route found? The blue line of Figure 4 says no: from 150 samples to 4,000 the median best route stays at 6.07 m.

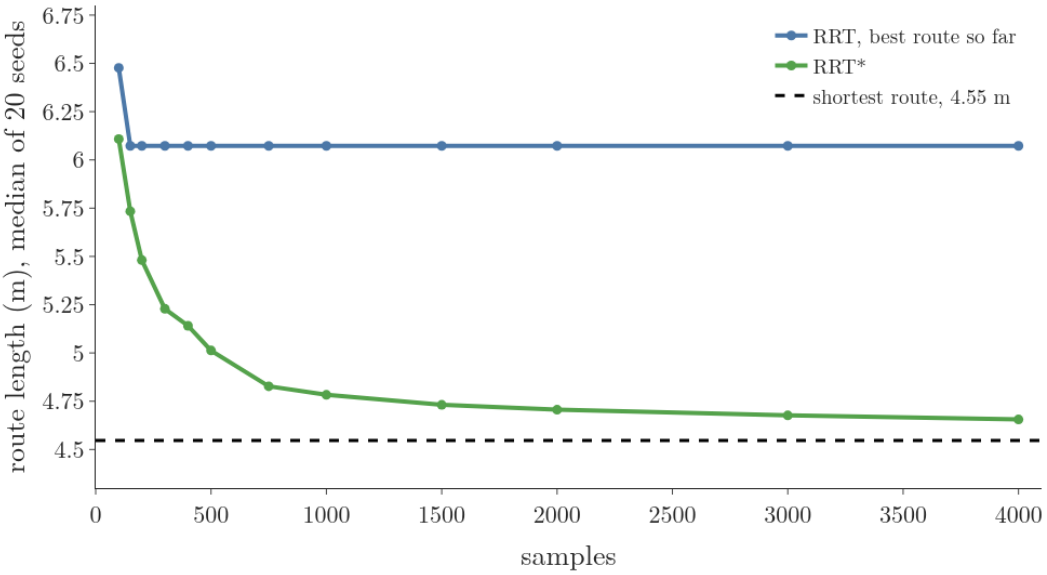


Figure 4: The best route's length against the number of samples, median of 20 runs from under the table: RRT, keeping the best route as the tree grows (blue), stays at 6.07 m from 150 samples on; RRT* (green) falls from 6.11 m at 100 samples to 5.01 m at 500, 4.71 m at 2,000 and 4.66 m at 4,000, towards the shortest route, 4.55 m (dashed)

The reason is in step 4 of the algorithm: a node's parent is fixed when the node is added, and a new node only ever attaches to the nearest node. Once the tree has a crooked branch to the door, later nodes add new branches but never straighten the old one. Karaman and Frazzoli (2011) proved that the length of RRT's route converges to a value above the shortest, with probability 1. The robot has a route, but a long one; the tree must be allowed to change its parents.

3. Shortening the route: RRT* and asymptotic optimality

> **Key point: RRT*** (G-2765) adds two moves to each round: the new node takes, as its parent, the nearby node that gives it the shortest route from the start, and then nearby nodes switch their parent to the new node when that shortens their route, **rewiring** (G-2766). The route from under the table falls from 6.61 m to 4.79 m over 2,000 samples, towards the shortest 4.55 m.

3.1 Choose the parent, then rewire

The route of Section 2.5 stays 6.07 m long because parents never change. Picture each node as a town that keeps the shortest known road back to the start. When a new town appears, it connects to whichever nearby town gives it the shortest road home, not just the closest one; then each nearby town checks whether going home through the new town would be shorter, and if so switches.

The standard terms. RRT with these two moves is **RRT*** (G-2765), and switching a node's parent to the new node is **rewiring** (G-2766) (Karaman and Frazzoli 2011, Algorithm 6; CMU 16-350 slides). Each node stores its cost: the length of its route from the start through the tree. "Nearby" means within a radius r, here 0.5 m.

Figure 5 zooms in on one round of an RRT* run, sample 119, left of the table. Watch the stages: the new node, the three nodes within 0.5 m, the choice of parent, then the rewiring of node C.

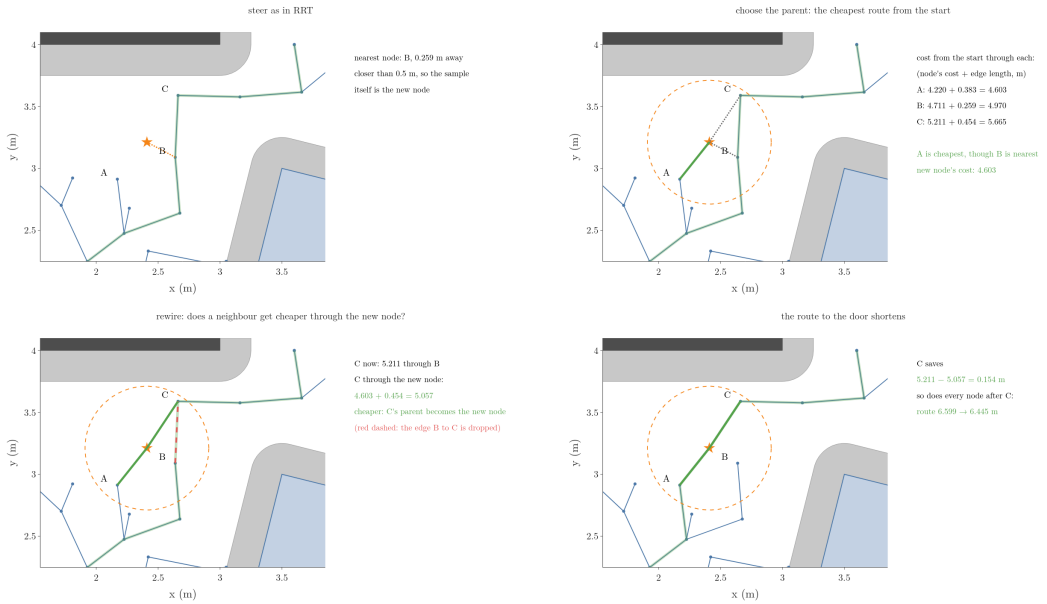


Figure 5: Animation: one RRT* round up close, sample 119 of the run of Figure 6, left of the table. The sample (2.410, 3.212) is within 0.5 m of its nearest node B, so it becomes the new node (orange star). Three nodes lie within r = 0.5 m (dashed circle): A, B and C. Choose the parent: the cost from the start through each, the node's cost plus the edge length: A 4.220 + 0.383 = 4.603, B 4.711 + 0.259 = 4.970, C 5.211 + 0.454 = 5.665; A is cheapest though B is nearest. Rewire: C costs 5.211 through its parent B but 4.603 + 0.454 = 5.057 through the new node, so its parent becomes the new node and the edge from B to C is dropped (red dashed). The route to the door shortens from 6.599 m to 6.445 m

Choosing the parent, on numbers (costs in m, rounded to millimetres and added as shown):

- $c(A) = 4.220$, $c(B) = 4.711$, $c(C) = 5.211$ are the three nodes' costs, their route lengths from the start;
- the edges from A, B and C to the new node are 0.383, 0.259 and 0.454 m.

The cost through each:

$$\text{through A} = 4.220 + 0.383 = 4.603$$

$$\text{through B} = 4.711 + 0.259 = 4.970$$

$$\text{through C} = 5.211 + 0.454 = 5.665$$

A gives the shortest route, so A becomes the parent, although B is the nearest node, and the new node's cost is 4.603. Plain RRT would have taken B, at 4.970.

Rewiring, on numbers. Node C's route through its parent B costs 5.211. Through the new node:

$$4.603 + 0.454 = 5.057$$

$$5.211 - 5.057 = 0.154 \text{ m shorter}$$

So C's parent becomes the new node. Every node after C on the way to the door gets 0.154 m shorter too: the route to the door falls from 6.599 m to 6.445 m.

The same in symbols. With:

- q_{new} the new node, and Q_{near} the set of tree nodes within r of it, here {A, B, C};
- q one node of Q_{near}, such as A;
- $c(q)$ the cost of node q, its route length from the start, in m;
- $\|q - q_{\text{new}}\|$ the length of the straight edge between q and q_{new}, in m.

The new node's parent is the q in Q_{near} with a free edge that gives the smallest cost:

$$c(q_{\text{new}}) = \min_{q} \Big(c(q) + \|q - q_{\text{new}}\| \Big)$$

Then each q in Q_{near} with a free edge is rewired when

$$c(q_{\text{new}}) + \|q_{\text{new}} - q\| < c(q)$$

and the costs of all nodes below q in the tree fall by the same amount (Karaman and Frazzoli 2011, Algorithm 6). Both moves only ever lower a cost, so the best route to the door can only get shorter.

3.2 The route shrinks as the tree grows

Figure 6 plays a whole RRT* run from under the table, one snapshot per frame from the first route at sample 63 to sample 2,000, with the best route's length plotted on the right. Watch the green route straighten and slide closer to the table's corner, towards the dashed shortest route.

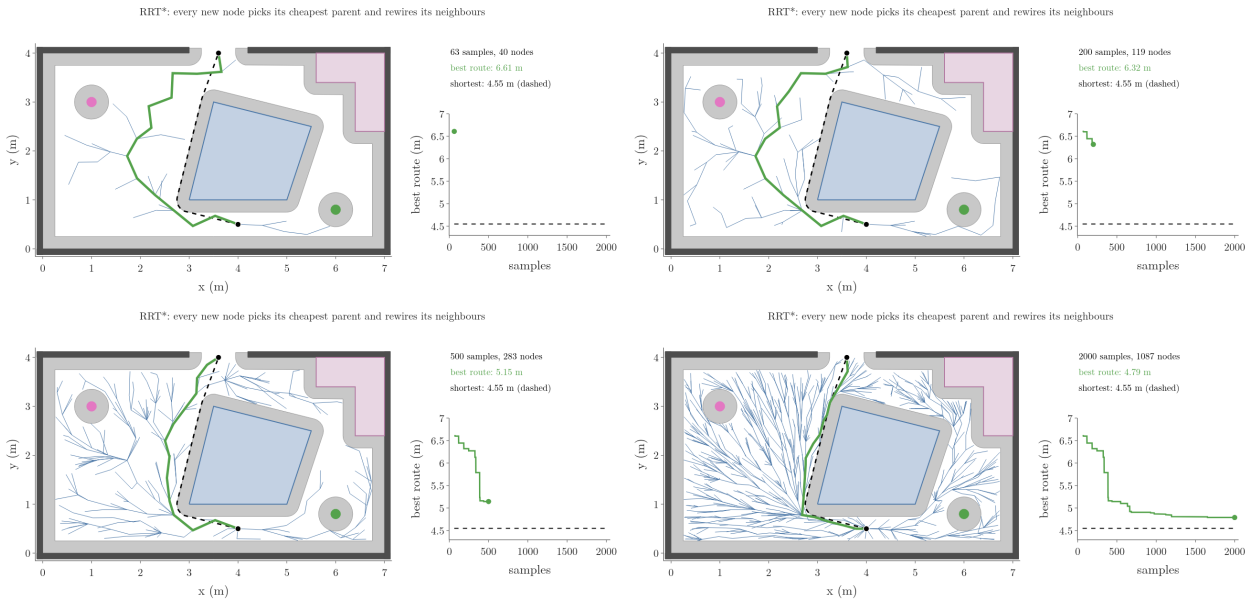


Figure 6: Animation: RRT* from under the table (4.0, 0.5) to the doorway centre, the tree after 63, 75, 100, 150, 200, 300, 400, 500, 700, 1,000, 1,500 and 2,000 samples. Green: the best route to the door so far; dashed: the shortest route, 4.55 m. Right: the best route's length against the number of samples. The first route appears at sample 63, 6.61 m; it is 6.32 m at 200 samples, 5.15 m at 500, 4.87 m at 1,000 and 4.79 m at 2,000, when the tree has 1,087 nodes

In Figure 6 the route falls from 6.61 m to 4.79 m. Over 20 runs (green line of Figure 4), the median route is 6.11 m at 100 samples, 5.01 m at 500, 4.71 m at 2,000 and 4.66 m at 4,000: each time closer to 4.55 m, where RRT stays at 6.07 m. RRT* answers the second part of the opening problem: with enough samples, the route gets as close to the shortest as we wish.

The extra moves cost collision checks: over 2,000 samples, RRT* made 11.9 checks per sample against RRT's 3.1, because every nearby node's edge is checked for the parent choice and for rewiring (notebook).

3.3 Asymptotic optimality

Section 3.2 showed the RRT* route falling towards 4.55 m while RRT's stays at 6.07 m. The property behind that difference has a name.

The standard term. A planner whose route length converges to the shortest possible, with probability 1, as the number of samples grows without end, is **asymptotically optimal** (G-2767) (Karaman and Frazzoli 2011, Definition 24). RRT* is asymptotically optimal; RRT is not, since its route converges to a longer value (Karaman and Frazzoli 2011). Figure 4 shows both on our room.

How large a radius. Rewiring only within a fixed small radius could miss the nodes that would shorten a route; checking every node would make each round slow. Karaman and

Frazzoli let the radius shrink as the tree grows, slowly enough that a new node still has about the logarithm of the number of nodes as neighbours. With:

- n is the number of nodes in the tree;
- d is the number of numbers in a configuration, 2 here, (x, y);
- γ (gamma) is a constant, in m: 4.0 here;
- η is the longest step, 0.5 m, which also caps the radius;
- ln is the natural logarithm.

$$r(n) = \min\left(\gamma\left(\frac{\ln n}{n}\right)^{1/d},\ \eta\right)$$

(Karaman and Frazzoli 2011, Algorithm 6.) With 1,000 nodes:

$$r = 4.0 \times \sqrt{\ln 1000/1000}$$

$$= 4.0 \times \sqrt{6.908/1000}$$

$$= 4.0 \times 0.0831 = 0.332 \text{ m}$$

Up to about 380 nodes the formula gives more than 0.5 m, so the radius is 0.5 m; at 2,000 nodes it is 0.247 m. For the guarantee, γ must exceed a bound set by the area of the free space (Karaman and Frazzoli 2011, Theorem 38). With:

- A is the area of free floor for the robot's centre, 15.02 m^2;
- $\gamma_{\min}$ is the bound, in m.

For $d = 2$:

$$\gamma_{\min} = \sqrt{3}\sqrt{A/\pi}$$

$$= \sqrt{3} \times \sqrt{15.02/3.1416}$$

$$= 1.7321 \times 2.1866 = 3.787 \text{ m}$$

so $\gamma = 4.0$ is enough.

Relatives, by name. The same idea gives other asymptotically optimal planners, named here only:

- **PRM*** (G-2769): a roadmap planner, which connects each sample to its neighbours instead of growing one tree, and tries more connections as the roadmap grows, so its routes converge to the shortest (Karaman and Frazzoli 2011; OMPL planners).

- **FMT*** (G-2770), the fast marching tree: it draws all its samples first, then grows a tree outwards in order of cost from the start, as Dijkstra's algorithm does, checking for collisions only when a connection is about to be made (OMPL planners).
- **SST*** (G-2771), from Stable Sparse-RRT: for robots with motion limits, whose steer cannot simply draw a straight line to a sample; it keeps only a sparse set of nodes and is asymptotically optimal, where SST is asymptotically near-optimal (Li, Littlefield and Bekris 2016; OMPL planners).

RRT* gets the route as short as we wish, given time. The first route, though, still took 63 samples to appear, from one tree that had to find its own way through the 0.7 m gap of the doorway.

4. Two trees that meet: RRT-Connect

> **Key point: RRT-Connect** (G-2768) grows one tree from the start and one from the goal; when one tree adds a node, the other tree steps towards that node again and again until it reaches it or hits an obstacle. From under the table the trees met after a median of 27 rounds, against 65 samples for RRT.

The third part of the opening problem: how soon can a first route appear? RRT's tree must find the doorway, whose free stretch for the robot's centre is only 0.7 m wide, from the far side of the table. Picture two search parties, one setting out from each end, each running towards whatever the other has just reached.

The standard term. Searching from both ends is bidirectional search (G-2584). Its tree version with a greedy connection step is **RRT-Connect** (G-2768) (Kuffner and LaValle 2000, as presented in the CMU 16-350 slides; LaValle §5.5.3). One round, with tree A and tree B:

1. Tree A takes one RRT step, at most 0.5 m, towards a random sample.
2. If that added a node, tree B CONNECTs to it: it takes RRT steps towards that new node, again and again, until it reaches it (the trees meet) or a step is blocked.
3. The trees swap roles for the next round.

The repeated steps of CONNECT let a tree cross open floor in one round, instead of 0.5 m per round (CMU 16-350 slides). The swap makes both trees explore.

Figure 7 plays a run. Blue grows from the start under the table, purple from the door. Watch the rounds where a tree adds a node and the other runs towards it.

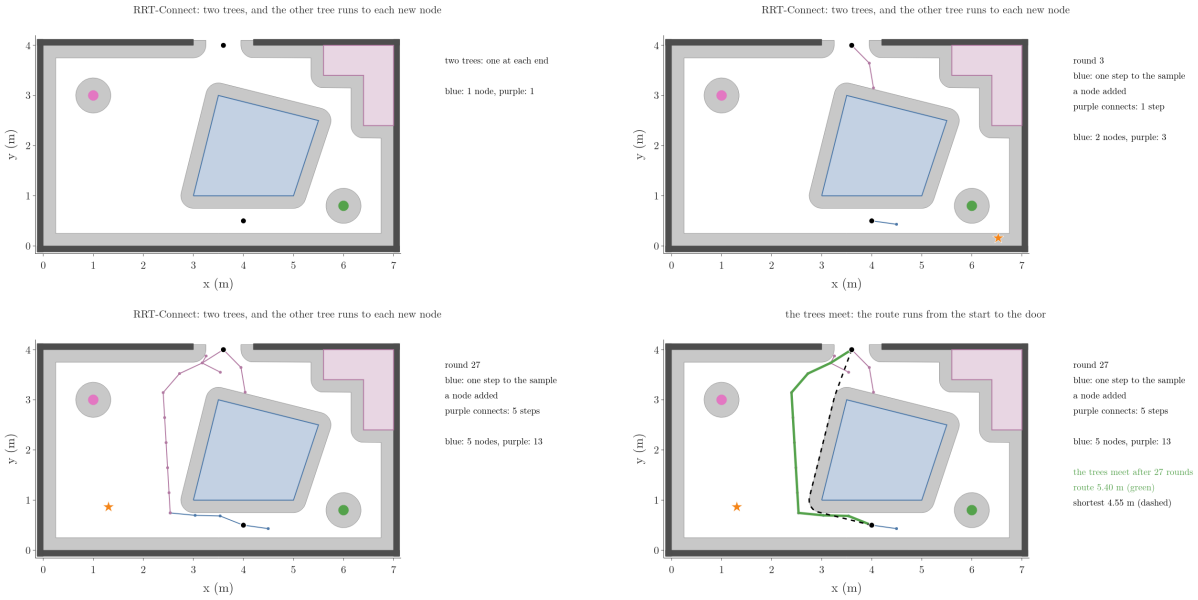


Figure 7: Animation: RRT-Connect from under the table (4.0, 0.5, blue tree) and the doorway centre (3.6, 4.0, purple tree), one round per frame. Each round one tree takes one step of at most 0.5 m towards a random sample (orange star); if it added a node, the other tree steps towards that node again and again until it reaches it or is blocked; then the trees swap. In round 27 the blue tree adds a node left of the table and the purple tree reaches it in 5 steps: the trees meet. Last frame: the route, 5.40 m (green), and the shortest route, 4.55 m (dashed)

In Figure 7 most rounds are blocked: the blue tree is boxed in the strip under the table, and the purple tree grows down the left of the table. In round 27 the blue tree's new node, left of the table, is in reach: the purple tree runs 5 steps to it and the trees meet. The route is 5.40 m. Over 200 runs, the trees met after a median of 27 rounds, with a median route of 5.61 m, against RRT's 65 samples and 6.07 m (notebook). Two trees answer the third part of the opening problem: they find the first route sooner, after 27 rounds instead of 65 samples (medians).

RRT-Connect keeps RRT's guarantee of finding a route if one exists, with probability approaching 1, but like RRT it is not asymptotically optimal: its first route is all it offers (CMU 16-350 slides). Its routes are usually shortened afterwards, by trying straight cuts between points of the route (CMU 16-350 slides).

5. Summary

Planner	Each round	From under the table	Route as samples grow
RRT	sample, nearest node, steer 0.5 m, check, add	65 samples, 6.07 m (median of 200)	stays at 6.07 m
RRT*	as RRT, then choose the cheapest parent and rewire within r(n)	first route at 63 samples (one run)	5.01 m at 500, 4.66 m at 4,000 (median of 20)
RRT-Connect	one tree steps to a sample, the other connects to the new node, swap	27 rounds, 5.61 m (median of 200)	not improved

- The robot under the table finds a route with no grid, and the route gets as close to the shortest as time allows: a **rapidly-exploring random tree** (G-2761) reached the door after 67 samples and 465 collision checks with a 6.08 m route, and RRT* had 4.79 m after 2,000 samples, against the shortest 4.55 m, because each sample needs only a nearest node and a checked 0.5 m edge, and rewiring keeps shortening the routes; a first route appears sooner with two trees, after a median of 27 rounds against 65 samples, because each tree runs towards the other's new nodes.
- A **sampling-based planner** (G-2760) replaces the grid's millions of cells with random configurations and collision checks, so it works where a grid cannot, such as six joints at 2.2×10^{15} cells.
- The **nearest node** (G-2762) to a random sample usually sits at the edge of the explored region, so RRT grows towards empty space: 95 percent of the floor within 0.5 m after 100 nodes, against 27 percent for a random-walk tree.
- **Steer** (G-2763) moves at most 0.5 m towards the sample, so every edge is short enough to check at 11 points.
- A **goal bias** (G-2764) of 0.05 makes one sample in twenty the goal itself, so the tree tries to finish without turning greedy.
- RRT's route stays at 6.07 m however long it runs, because a node's parent never changes; **RRT*** (G-2765) picks the cheapest nearby parent and **rewires** (G-2766) neighbours through the new node, so the route only ever shortens: 4.79 m after 2,000 samples in one run.
- RRT* is **asymptotically optimal** (G-2767), its route converging to the shortest with probability 1, provided its radius shrinks no faster than about the square root of ln n / n in two dimensions; RRT is not.
- **PRM*** (G-2769), **FMT*** (G-2770) and **SST*** (G-2771) are asymptotically optimal relatives: a roadmap, a tree grown in cost order, and a sparse tree for robots with motion limits; they are named here because planner libraries such as OMPL offer them beside RRT*.

- **RRT-Connect** (G-2768) grows a tree from each end and lets each run to the other's new nodes, so the trees met after 27 rounds instead of 65 samples, though its route is not shortened.

The robot can now plan from under the table to the door with only collision checks and nearest neighbours, and RRT* makes the route as short as time allows. Still missing: each tree serves one start and one goal and is thrown away after, and a narrow passage starves uniform samples; a roadmap of random samples built once and searched for many queries is the next step, probabilistic roadmaps.

6. Sources

Built from

- MATLAB, "Path Planning with A* and RRT | Autonomous Navigation, Part 4", YouTube, https://www.youtube.com/watch?v=QR3U1dgc5RE
- Northwestern Robotics, "Modern Robotics, Chapter 10.5: Sampling Methods for Motion Planning (Part 2 of 2)", YouTube, https://www.youtube.com/watch?v=Ao7p_xiUu4s, with its narration on the book's resource page, https://modernrobotics.northwestern.edu/nu-gm-book-resource/10-5-sampling-methods-for-motion-planning-part-2-of-2/
- Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press. §10.5 "Sampling Methods for Motion Planning", §10.5.1 "The RRT Algorithm" (Algorithm 10.3, Figure 10.17 on RRT against a random tree, the sampler, the nearest node, the local planner, bidirectional RRT, RRT*). Free preprint: http://modernrobotics.org (MR §10.5, §10.5.1)
- Likhachev, M. (Carnegie Mellon University), *16-350 Planning Techniques for Robotics*, slides "RRT, RRT-Connect, RRT*", https://www.cs.cmu.edu/~maxim/classes/robotplanning/lectures (CMU 16-350 slides)
- LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press. Ch.5 "Sampling-Based Motion Planning": §5.4.3 (the 50^{10} example), https://lavalle.pl/planning/node225.htm §5.5 "Rapidly Exploring Dense Trees", https://lavalle.pl/planning/node230.html; §5.5.1 "The Exploration Algorithm", https://lavalle.pl/planning/node231.html; §5.5.3 "Using the Trees for Planning" (single-tree search, balanced bidirectional search), https://lavalle.pl/planning/node236.html and https://lavalle.pl/planning/node237.html (LaValle ch.5, §5.4.3, §5.5, §5.5.1, §5.5.3)
- Karaman, S. and Frazzoli, E. (2011). "Sampling-based algorithms for optimal motion planning". *The International Journal of Robotics Research* 30(7), 846–894. arXiv:1105.1186, https://arxiv.org/abs/1105.1186 (Karaman and Frazzoli 2011)
- OMPL, "Available Planners", https://ompl.kavrakilab.org/planners.html (OMPL planners)

Other references

- Kuffner, J. J. and LaValle, S. M. (2000). "RRT-Connect: An efficient approach to single-query path planning". *Proceedings of the IEEE International Conference on Robotics and Automation*, as presented in the CMU 16-350 slides

- Li, Y., Littlefield, Z. and Bekris, K. E. (2016). "Asymptotically optimal sampling-based kinodynamic planning". *The International Journal of Robotics Research* 35(5), 528–564. arXiv:1407.2896, https://arxiv.org/abs/1407.2896 (Li, Littlefield and Bekris 2016)

7. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Sampling-based planner (G-2760)	A motion planner that never builds the obstacle region or a grid, but draws random configurations and tests them with collision checks, joining the free ones into a tree or graph; it works in C-spaces with many dimensions, where grids are far too large.
Rapidly-exploring random tree (RRT) (G-2761)	A sampling-based planner that grows a tree from the start: each round it draws a random configuration, steps the tree's nearest node at most a fixed distance towards it and keeps the new node if the step is free, until the tree reaches the goal; it explores fast but its routes zig-zag.
Nearest node (G-2762)	The node of a planner's tree closest to a new sample under the planner's distance; RRT extends it, which makes the tree grow towards the largest empty regions.
Steer (steering function) (G-2763)	The step of a sampling-based planner that moves from a tree node towards a sample, at most a fixed distance (0.5 m here), stopping on the sample if it is closer; it keeps every new edge short enough to check.
Goal bias (G-2764)	The small probability (0.05 here) with which a sampling-based planner uses the goal itself as its next sample, so the tree keeps trying to reach the goal; too large a bias makes it greedy, too small leaves it wandering.
RRT* (G-2765)	RRT with two extra moves per round: the new node takes as parent the nearby node that gives it the shortest route from the start, and nearby nodes are rewired through it when that is shorter; its route converges to the shortest as samples grow.

Term	Meaning
Rewiring (G-2766)	Switching a tree node's parent to a newly added node when the route from the start through the new node is shorter; it lowers the costs of the node and everything below it, so the tree's routes only shorten.
Asymptotically optimal (G-2767)	Said of a sampling-based planner whose route length converges to the shortest possible, with probability 1, as the number of samples grows without end, such as RRT*; plain RRT is not.
PRM* (G-2769)	An asymptotically optimal probabilistic roadmap: it connects each sample to its neighbours and tries more connections as the roadmap grows, so its routes converge to the shortest.
FMT* (fast marching tree) (G-2770)	An asymptotically optimal sampling-based planner that draws all its samples first, then grows a tree outwards in order of cost from the start, checking for collisions only when a connection is about to be made.
SST* (Stable Sparse-RRT) (G-2771)	An asymptotically optimal tree planner for robots with motion limits, whose steer cannot draw an exact path to a sample; it keeps only a sparse set of nodes. Its sibling SST is asymptotically near-optimal.
RRT-Connect (bidirectional RRT) (G-2768)	An RRT that grows one tree from the start and one from the goal; when one tree adds a node, the other steps towards it again and again until it reaches it or is blocked, then the trees swap roles; it finds a first route sooner but does not shorten it.
Potential field (artificial potential field) (G-2740)	A height over the robot's C-space that is lowest at the goal and rises near obstacles, built as an attractive bowl plus a repulsive hill round each obstacle; the robot reaches the goal by stepping downhill from wherever it is, with no grid and no search, but can stop in a local minimum.
Local minimum (G-1110)	A point lower than everything around it, but not the lowest overall.

Term	Meaning
Graph (G-2540)	A set of vertices (points) joined by edges (lines), such as the room's 76 free cells joined by 114 moves; it keeps only which places connect directly, so one search method works for maps, puzzles and any problem of states and moves.
Parent (back pointer) (G-2576)	The vertex from which a search reached a vertex, stored with it during the search; following parents back from the goal gives the route. In A*, a cell's parent changes whenever a shorter way in lowers its cost so far.
Euclidean distance (G-715)	The straight-line distance between two points.
Edge resolution (segment checking) (G-2679)	Checking a straight path segment in C-space for collisions by testing points along it, spaced at most a chosen step apart; too long a step can miss a thin collision, too short a step wastes time.
Randomized potential field (G-2744)	A planner that follows a potential field downhill and, when stuck in a local minimum, takes random walks until it finds a lower point, then descends again, backtracking after many failed walks; it escapes local minima, at the price of long paths and many tuned settings.
Metric (distance function) (G-2681)	A distance between points that is never negative, is 0 only between equal points, is the same both ways and obeys the triangle inequality; search trees rely on these rules to skip far regions safely.
Nearest-neighbour search with wrap-around (G-2685)	A k-d tree search that measures a cell's distance from the query the short way round the circle of headings, so a cell just across 0° is not skipped and the true nearest configuration is found.
Bidirectional search (G-2584)	A forward search from the start and a backward search from the goal run in turn until some state has been expanded by both; the route is the smallest sum of the two costs over all states both reached, since the meeting state can be off the shortest route.